



Universidad de las Fuerzas Armadas
ESPE

Documento de Arquitectura de Referencia

Sistema de Gestión Académica
Proyecto automatización de notas

Versión 1.0.0

https://github.com/Fernach0/notas_v2

Bryan Quispe y Luis Cueva

Información General

Título:	Documento de Arquitectura de Referencia
Subtítulo:	Sistema de Gestión Académica — notas_v2
Versión:	1.0.0
Repositorio:	https://github.com/Fernach0/notas_v2
Autor:	[Tu Nombre]
Estado:	En Desarrollo

Firmas y Aprobaciones

Elaborado Por:	[Bryan Quispe , Luis Cueva] <i>Cargo: Desarrollador / Autor</i>	Fecha: [20/04/2026] Firma: _____
Revisado Por:	[Nombre] <i>Cargo: [Cargo]</i>	Fecha: [] Firma: _____
Aprobado Por:	[Nombre] <i>Cargo: [Cargo]</i>	Fecha: [] Firma: _____

Lista de Cambios

Versión	Fecha	Autor	Descripción
1.0.0	[20/04/2026]	[Bryan Quispe , Luis Cueva]	Emisión Inicial

1 Introducción

Este documento describe la arquitectura del sistema notas_v2, un sistema de gestión académica desarrollado para instituciones educativas de Ecuador. El proyecto está publicado en el repositorio https://github.com/Fernach0/notas_v2 y consiste en una API REST construida con NestJS y un frontend desarrollado con Next.js 15.

Su propósito es capturar y comunicar las decisiones arquitectónicas más significativas tomadas durante el diseño e implementación del sistema, sirviendo como referencia para desarrolladores, revisores y equipos que participen en el mantenimiento o extensión del proyecto.

1.1 Objetivo

Describir en detalle la arquitectura base del sistema notas_v2: sus capas, módulos, modelos de datos, patrones de diseño aplicados, mecanismos de seguridad, flujos de interacción y estrategia de despliegue, con el nivel de detalle suficiente para que un equipo de desarrollo pueda entender, mantener y extender el sistema.

1.2 Justificación

La arquitectura se definió con base en los siguientes lineamientos:

- Apego a estándares: Se utilizan tecnologías con amplio respaldo de la industria (NestJS, Prisma, Next.js, PostgreSQL), garantizando soporte a largo plazo y comunidad activa.
- Separación de responsabilidades: Backend y frontend son proyectos independientes. El backend expone una API REST agnóstica del cliente; el frontend la consume vía HTTP con JWT.
- Seguridad por capas: Autenticación JWT, rate-limiting global, hash de contraseñas con bcrypt, validación estricta de DTOs en todos los endpoints, y guards de rol diferenciados.
- Trazabilidad académica: El modelo de datos soporta múltiples años lectivos, parciales, calificaciones individuales y en masa, promedios automáticos y evidencias PDF por actividad.
- Productividad del desarrollador: Swagger autogenerado, seed de datos de prueba, estructura modular de NestJS y servicios tipados en el frontend reducen la curva de aprendizaje.
- Escalabilidad: La arquitectura por módulos permite añadir funcionalidades (mensajería, reportes, integración con sistemas externos) sin modificar el núcleo existente.

1.3 Alcance

Este documento cubre el backend (NestJS + Prisma + PostgreSQL), el frontend (Next.js 15 + TanStack Query + NextAuth.js), el modelo de datos completo, la API REST con sus 14 grupos de endpoints, los mecanismos de seguridad, el flujo de autenticación, la gestión de evidencias PDF y la arquitectura de despliegue del proyecto notas_v2.

1.4 Referencias

- Repositorio: https://github.com/Fernach0/notas_v2
- Documentación de endpoints: /backend/ENDPOINTS.md
- Plan de acción del backend: /Planes de accion/PLAN_DE_ACCION_1.md
- Plan de acción del frontend: /frontend/PLAN_FRONTEND.md
- NestJS Documentation: <https://docs.nestjs.com>
- Prisma Documentation: <https://www.prisma.io/docs>
- Next.js 15 App Router: <https://nextjs.org/docs/app>

- NextAuth.js v5 (Auth.js): <https://authjs.dev>
- TanStack Query v5: <https://tanstack.com/query/v5>
- Swagger UI (API Docs): <http://localhost:3000/api/docs>

2 Arquitectura del Sistema

El sistema notas_v2 sigue una arquitectura cliente-servidor de tres capas lógicas bien definidas: presentación (frontend Next.js), lógica de negocio (backend NestJS) y persistencia (PostgreSQL vía Prisma). La comunicación entre frontend y backend se realiza exclusivamente mediante HTTP/REST con tokens JWT para la autorización.

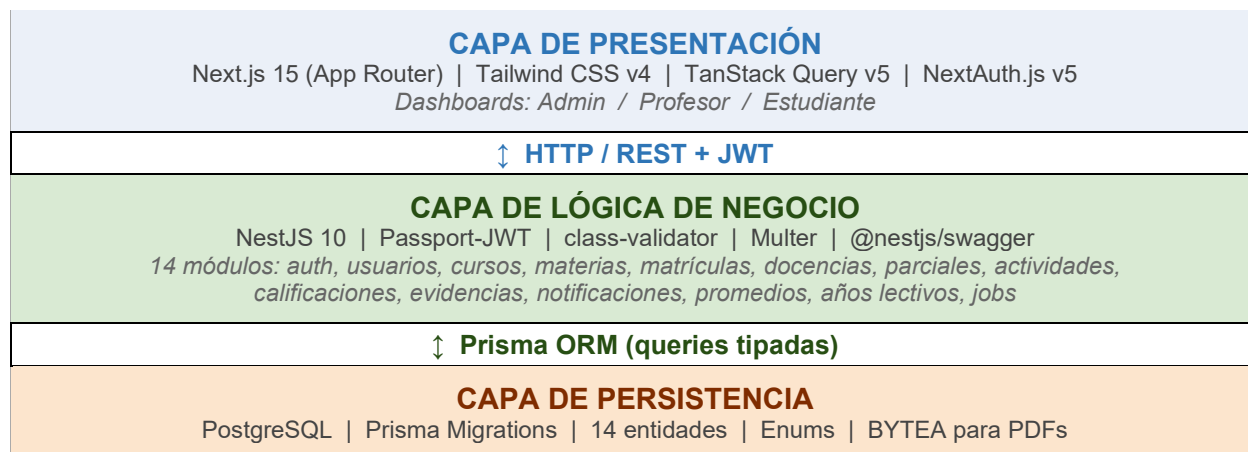
Desde el punto de vista físico, el sistema puede desplegarse en dos nodos independientes: el servidor de la API (Node.js) y la base de datos (PostgreSQL). El frontend puede desplegarse como un servidor Next.js separado o como una aplicación estática exportada.

La arquitectura en capas lógicas del sistema se describe a continuación:

- **Capa de Presentación (Frontend):** Interfaz web construida con Next.js 15 (App Router). Soporta tres dashboards diferenciados por rol: Administrador, Profesor y Estudiante. Las rutas están protegidas por Middleware de Next.js que verifica la sesión y el rol antes de renderizar cualquier página.
- **Capa de Control / API (Backend):** API REST implementada con NestJS 10. Organizada en 14 módulos funcionales, cada uno con su propio controlador, servicio y DTOs. Expone una documentación Swagger interactiva en /api/docs.
- **Capa de Persistencia:** PostgreSQL como motor de base de datos relacional. Prisma ORM gestiona el esquema (schema.prisma), las migraciones y las consultas tipadas. El modelo contempla 14 entidades con sus relaciones e índices únicos.

Un aspecto relevante de la arquitectura es la gestión de evidencias: los archivos PDF subidos por los estudiantes se almacenan directamente en la base de datos en un campo BYTEA (bytes). Un Job programado con @nestjs/schedule se encarga de la retención: purga los PDFs de años lectivos finalizados mediante una operación de soft-delete.

2.1 Diagrama lógico de capas



3 Patrones de la Arquitectura

Para la definición de la arquitectura del sistema se han aplicado los siguientes patrones:

- Modelo-Vista-Controlador (MVC)
- Data Access Object (DAO) — implementado mediante Prisma y la capa de Servicios
- Repository Pattern — encapsulado en cada módulo de NestJS
- Data Transfer Object (DTO) — validación de entradas con class-validator
- Guard / Interceptor Pattern — seguridad declarativa con Guards de NestJS

3.1 Modelo-Vista-Controlador (MVC)

El patrón MVC estructura la aplicación separando los datos (Modelo — Prisma/PostgreSQL), la presentación (Vista — Next.js con React) y la lógica de control (Controlador — NestJS). Cada módulo de NestJS implementa el Controlador, que recibe la solicitud HTTP, la valida mediante DTOs y delega la lógica al Servicio. El Servicio accede al Modelo a través de Prisma y retorna los datos al Controlador, que los serializa como respuesta JSON. El frontend (Vista) consume esta respuesta a través de TanStack Query y la presenta al usuario mediante componentes React.

3.2 Data Access Object (DAO)

El patrón DAO se implementa en la capa de Servicios de NestJS. Cada servicio (ej. UsuariosService, CalificacionesService) encapsula todas las operaciones de acceso a datos para su entidad correspondiente, utilizando el PrismaService inyectado. Esto desacopla la lógica de negocio del mecanismo de persistencia, permitiendo cambiar de ORM o base de datos sin afectar los controladores ni la lógica de alto nivel.

3.3 Data Transfer Object (DTO)

Todos los datos de entrada a la API son validados mediante DTOs contruidos con class-validator y class-transformer. El ValidationPipe global (configurado en main.ts con whitelist: true y forbidNonWhitelisted: true) rechaza cualquier propiedad no declarada en el DTO y transforma automáticamente los tipos primitivos. Esto garantiza que ningún dato malformado llegue a la capa de servicios. Un caso especial es la validación de la cédula ecuatoriana: el campo idUsuario implementa un validador personalizado (cedula.validator.ts) que verifica el algoritmo del dígito verificador antes de aceptar el registro.

3.4 Guard Pattern (Autorización por Roles)

La autorización se implementa mediante dos Guards declarativos: JwtAuthGuard (verifica que el token JWT sea válido y no haya expirado) y RolesGuard (verifica que el usuario autenticado tenga el rol requerido por el endpoint). Los roles se declaran mediante el decorador @Roles() a nivel de controlador o método. Esta combinación permite proteger rutas de forma granular sin repetir lógica de seguridad en cada servicio.

4 Diseño de la Arquitectura

4.1 Stack Tecnológico

La siguiente tabla resume las tecnologías utilizadas en cada capa del sistema y su rol específico:

Capa	Tecnología	Rol en el sistema
Backend	NestJS 10 (Node.js)	Framework modular con soporte nativo para DI, guards, pipes y decoradores.
ORM / BD	Prisma + PostgreSQL	Mapeo objeto-relacional tipado. Migraciones declarativas vía schema.prisma.
Autenticación	passport-jwt + @nestjs/jwt	JWT firmado con HS256. Expiración configurable (default: 8h).
Seguridad	bcrypt + ThrottlerGuard	Hash de contraseñas y rate-limiting (60 req/min por IP).
Validación	class-validator + class-transformer	Validación de DTOs en capa de entrada con ValidationPipe global.
Documentación API	@nestjs/swagger	Swagger UI autogenerado en /api/docs con autenticación Bearer.
Subida de archivos	Multer (multipart/form-data)	PDFs hasta 10 MB. Bytes almacenados en campo BYTEA de PostgreSQL.
Email	nodemailer	Envío de tokens de recuperación de contraseña.
Jobs	@nestjs/schedule (Cron)	Job de retención de evidencias: purga PDFs de años lectivos cerrados.
Frontend	Next.js 15 (App Router)	SSR/SSG con React 19. Rutas protegidas vía Middleware de Next.js.
Estilos	Tailwind CSS v4	Utility-first. Sin archivo de configuración (v4).
Estado / Fetching	TanStack Query v5 + Axios	Cache automático, revalidación y mutaciones. JWT inyectado vía interceptores.
Autenticación FE	NextAuth.js v5 (Auth.js)	Proveedor Credentials. JWT del backend persistido en sesión cifrada.
Formularios FE	React Hook Form + Zod	Validación de esquemas. Reutilización entre frontend y DTOs del backend.

4.2 Estructura del Repositorio

El repositorio notas_v2 organiza el código en dos proyectos raíz: backend/ y frontend/. Adicionalmente, la carpeta Planes de acción/ documenta las decisiones de diseño por fases.

- notas_v2/backend/proyecto-notas-api/ — Proyecto NestJS. Contiene src/modules/ (14 módulos), src/common/ (guards, decoradores, validadores), src/prisma/ (PrismaModule y PrismaService), src/jobs/ y prisma/schema.prisma con las migraciones.

- `notas_v2/frontend/notas-frontend/` — Proyecto Next.js 15. Contiene `app/` con Route Groups por rol: `(auth)/` para login y `(dashboard)/` con sub-rutas `admin/`, `profesor/` y `estudiante/`. Incluye `components/`, `services/`, `hooks/`, `lib/` y `types/`.
- `notas_v2/Planes de accion/` — Documentos Markdown con el plan de acción del backend (`PLAN_DE_ACCION_1.md`), correcciones al esquema SQL (`fix_1.md`, `fix_2.md`) y el plan del frontend (`PLAN_FRONTEND.md`).
- `notas_v2/backend/ENDPOINTS.md` — Documentación detallada de todos los endpoints con ejemplos de payload JSON y flujo de prueba completo (14 pasos).

4.3 Módulos del Backend

El backend está organizado en 14 módulos de NestJS, cada uno con responsabilidad única:

Módulo	Responsabilidad	Características clave
auth	POST /login, forgot-password, reset-password, change-password	JWT (passport-jwt), bcrypt, nodemailer
usuarios	CRUD completo + asignación de roles	ThrottlerGuard, validación cédula ecuatoriana
anio-lectivo	CRUD + cambio de estado (ACTIVO/FINALIZADO/PLANIFICADO)	Filtro por estado
cursos	CRUD + asignación/desasignación de materias	Relación con AnioLectivo
materias	CRUD básico	Nombre único por materia
matricula	Matricular / desmatricular estudiantes en cursos	Restricción: un curso por estudiante
docencia	Asignar / quitar profesores a materia+curso	Tabla PROFESOR_MATERIA_CURSO
parciales	CRUD + creación bulk (3 parciales a la vez)	numeroParcial: 1, 2 o 3
actividades	CRUD completo con fechas de entrega y valorMaximo	tipoActividad: TAREA/EXAMEN/PROYECTO/LECCION
calificaciones	Calificación individual y bulk por curso	UNIQUE (idUserario, idActividad)
evidencias	Subida PDF (multipart), descarga, soft-delete	Almacenamiento en bytes; máx 10 MB
notificaciones	Listar, marcar leída, marcar todas leídas, eliminar	Tipos: NUEVA_ACTIVIDAD, CALIFICACION, etc.
promedios	Recalcular promedio por materia/general, ranking de curso	Parciales 1, 2, 3 + promedio final
jobs	Job de retención: elimina PDFs de años lectivos cerrados	@nestjs/schedule (CronJob)

4.4 Modelo de Datos

El esquema de base de datos define 14 entidades relacionales gestionadas mediante Prisma. Las principales decisiones de diseño aplicadas al esquema original incluyen: conversión de estados a enums (EstadoUsuario, EstadoLectivo, EstadoEvidencia, TipoActividad), adición de restricciones UNIQUE en calificaciones y promedios, e incorporación de la tabla ProfesorMateriaCurso (que no existía en el esquema original) para modelar correctamente la relación docente-materia-curso.

Modelo (tabla BD)	Descripción
AnioLectivo	Año escolar con fechas de inicio/fin y estado del período lectivo.
Usuario	Persona del sistema (admin, profesor, estudiante). Identificado por cédula (10 dígitos).
Rol	Catálogo fijo: 1=ADMIN, 2=PROFESOR, 3=ESTUDIANTE. Relación M:N con Usuario.
Curso	Grupo académico (ej. «8vo EGB A») perteneciente a un AnioLectivo.
Materia	Asignatura (ej. «Matemáticas»). Se asigna a un Curso.
UsuarioCurso	Matrícula: relación entre Estudiante y Curso. Un estudiante puede estar en un solo curso.
ProfesorMateriaCurso	Docencia: asigna un Profesor a una Materia dentro de un Curso específico.
Parcial	Período de evaluación dentro de una Materia/Curso. Números 1, 2 o 3.
Actividad	Tarea, examen, proyecto o lección con fechas límite y valor máximo.
Calificacion	Nota (Float) que un Estudiante obtiene en una Actividad. Única por estudiante/actividad.
Evidencia	Archivo PDF subido por el estudiante como evidencia de una Actividad (almacenado en bytes).
Notificacion	Aviso generado al estudiante sobre nuevas actividades o calificaciones.
PromedioMateriaEstudiante	Promedios parciales (1,2,3) y final por materia/estudiante/año.
PromedioGeneralEstudiante	Promedio general y comportamiento del estudiante en un Curso.

4.5 API REST — Resumen de Endpoints

La API expone su prefijo global en /api y documenta todos sus endpoints con Swagger en /api/docs. A continuación se muestra un resumen de los 14 grupos de recursos disponibles:

Recurso	Endpoints disponibles
Auth	POST /api/auth/login, /forgot-password, /reset-password, /change-password
Usuarios	CRUD /api/usuarios + roles: POST/DELETE /api/usuarios/:id/roles
Años Lectivos	CRUD /api/anios-lectivos
Cursos	CRUD /api/cursos + materias: POST/DELETE /api/cursos/:id/materias

Materias	CRUD /api/materias
Matrículas	POST/DELETE /api/matriculas
Docencias	POST/GET/DELETE /api/docencias
Parciales	CRUD /api/parciales + bulk: POST /api/parciales/bulk
Actividades	CRUD /api/actividades
Calificaciones	CRUD /api/calificaciones + bulk + por estudiante
Evidencias	POST/GET/DELETE + descarga /api/evidencias/:id/descargar
Notificaciones	GET/PATCH (leer/leer-todas)/DELETE /api/notificaciones
Promedios	POST recalcular materia/general + GET + ranking
Jobs	POST ejecutar + GET estado /api/jobs/retencion-evidencias

5 Arquitectura del Frontend

5.1 Estructura de Rutas (App Router)

El frontend utiliza el App Router de Next.js 15 con Route Groups para separar el layout de autenticación del layout del dashboard sin afectar las URLs. La estructura de rutas es la siguiente:

- `app/(auth)/login/` — Página de inicio de sesión. Sin layout de dashboard. Accesible sin autenticación.
- `app/(dashboard)/layout.tsx` — Layout compartido con Navbar y Sidebar. Solo accesible con sesión activa.
- `app/(dashboard)/admin/` — Dashboard del Administrador. Sub-rutas: `usuarios/`, `anios-lectivos/`, `cursos/`, `materias/`.
- `app/(dashboard)/profesor/` — Dashboard del Profesor. Sub-rutas: `mis-cursos/`, `actividades/`, `calificaciones/`.
- `app/(dashboard)/estudiante/` — Dashboard del Estudiante. Sub-rutas: `mis-materias/`, `actividades/`, `evidencias/`.
- `app/api/auth/[...nextauth]/route.ts` — Handler de NextAuth.js para gestionar la sesión JWT.

5.2 Flujo de Autenticación

El flujo de autenticación sigue los pasos descritos a continuación:

- El usuario ingresa email y contraseña en la página `/login`.
- NextAuth.js llama al proveedor Credentials, que hace POST `/api/auth/login` al backend NestJS.
- El backend valida las credenciales, las hash con bcrypt y retorna `{ access_token, idUsuario, nombreCompleto, roles }`.
- NextAuth persiste `access_token`, `idUsuario` y roles en una cookie de sesión cifrada (JWT firmado con `NEXTAUTH_SECRET`).
- El Middleware de Next.js (`middleware.ts`) lee la sesión en cada request e intercepta el acceso si: el usuario no está autenticado, o intenta acceder a una ruta de un rol que no le corresponde.
- Axios inyecta el `access_token` del backend en el header `Authorization: Bearer {token}` de todas las llamadas a la API mediante un interceptor de request.
- Si el backend retorna 401, el interceptor de response de Axios ejecuta `signOut()` de NextAuth, limpiando la sesión y redirigiendo al login.

5.3 Gestión de Estado y Data Fetching

El estado del servidor se gestiona con TanStack Query v5, que provee caché automático, revalidación y estados de loading/error. Cada módulo de datos tiene su propio conjunto de queryKeys para que las mutaciones (crear, editar, eliminar) puedan invalidar selectivamente el caché.

El estado de la UI (apertura/cierre de modales, notificaciones toast) se gestiona con hooks de React locales (`useModal`, `useToast`) sin necesidad de una librería de estado global como Zustand o Redux.

Los formularios de creación y edición de datos utilizan React Hook Form para gestión del estado del formulario y Zod para validación de esquemas. El esquema Zod del campo cédula (`idUsuario`) replica el algoritmo del dígito verificador ecuatoriano.

6 Arquitectura para Deployment

La arquitectura de despliegue del sistema notas_v2 contempla dos servicios independientes: el servidor de la API (NestJS) y el servidor del frontend (Next.js). Ambos pueden ejecutarse en el mismo host o en servidores separados.

6.1 Variables de Entorno

El backend requiere las siguientes variables de entorno definidas en `.env`:

- `DATABASE_URL`: cadena de conexión a PostgreSQL (ej: `postgresql://user:pass@host:5432/notas_escuela`).
- `JWT_SECRET`: clave secreta para firma de tokens JWT. Debe ser larga y aleatoria en producción.
- `JWT_EXPIRES_IN`: tiempo de vida del token (default: 8h).
- `STORAGE_PATH`: ruta del directorio para PDFs (si se usa almacenamiento en disco como alternativa al BYTEA).
- `PORT`: puerto del servidor NestJS (default: 3000).

El frontend requiere las siguientes variables de entorno en `.env.local`:

- `NEXTAUTH_URL`: URL base del frontend (ej: `http://localhost:3001` en desarrollo).
- `NEXTAUTH_SECRET`: clave secreta para cifrar la cookie de sesión de NextAuth.
- `NEXT_PUBLIC_API_URL`: URL base de la API del backend (ej: `http://localhost:3000/api`).

6.2 Proceso de Despliegue

El proceso recomendado de despliegue es el siguiente:

- Base de datos: Levantar una instancia de PostgreSQL y crear la base de datos `notas_escuela`. Ejecutar `npx prisma migrate deploy` para aplicar las migraciones. Opcionalmente ejecutar `npx prisma db seed` para poblar datos de prueba.
- Backend: Construir con `npm run build` (genera `dist/`). Ejecutar `node dist/main.js` o usar PM2 para producción. El servidor escucha en `PORT` (default 3000). Verificar Swagger en `/api/docs`.
- Frontend: Construir con `npm run build`. Ejecutar `npm start` para producción (servidor Next.js). El servidor escucha en el puerto configurado (default 3001 para no colisionar con el backend).
- Control de versiones: Todo cambio al código pasa por Pull Request a la rama `main` del repositorio en GitHub, garantizando revisión de código antes de integración.

6.3 Job de Retención de Evidencias

El módulo `jobs` implementa un `CronJob` programado con `@nestjs/schedule` que permite al administrador ejecutar manualmente la purga de PDFs de años lectivos cerrados. El endpoint `POST /api/jobs/retencion-evidencias/ejecutar` recibe el `idAnioLectivo` y una cadena de confirmación explícita ("`ELIMINAR_PDFS_ANIO_N`"), realizando un `soft-delete` de las evidencias asociadas. El estado del último job puede consultarse en `GET /api/jobs/retencion-evidencias/ultimo-estado`.

7 Mecanismos de Arquitectura

7.1 Seguridad

Autenticación: Los usuarios se autentican mediante email y contraseña. El backend valida las credenciales comparando el hash bcrypt almacenado y retorna un JWT firmado con HS256. El token incluye idUsuario y roles en el payload.

Autorización: Cada endpoint protegido requiere un JWT válido (JwtAuthGuard) y opcionalmente uno o más roles específicos (RolesGuard + @Roles() decorator). Los endpoints del administrador requieren rol 1, los del profesor rol 2 y los del estudiante rol 3.

Rate Limiting: ThrottlerGuard aplicado globalmente limita cada IP a 60 solicitudes por minuto, protegiendo contra ataques de fuerza bruta.

Hash de contraseñas: Todas las contraseñas se almacenan como hash bcrypt. Nunca se almacena la contraseña en texto plano.

Recuperación de contraseña: Genera un UUID como token de recuperación, lo almacena con fecha de expiración en el usuario, y lo envía al email registrado mediante nodemailer. El token se invalida tras su uso.

7.2 Validación y Transformación de Datos

El ValidationPipe global de NestJS aplica validación estricta sobre todos los DTOs: whitelist (elimina propiedades no declaradas), forbidNonWhitelisted (retorna 400 si hay propiedades extra) y transform (convierte strings a numbers/booleans según el tipo TypeScript). El campo cédula ecuatoriana (idUsuario) tiene un validador personalizado (CedulaValidator) que implementa el algoritmo del Ministerio de Telecomunicaciones del Ecuador.

7.3 Gestión de Promedios

Los promedios no se calculan en tiempo real sino bajo demanda: el profesor o el sistema invoca explícitamente los endpoints de recálculo. El módulo promedios calcula el promedio de cada parcial (promedio de todas las actividades del parcial), el promedio final de materia (promedio de los 3 parciales) y el promedio general del estudiante en el curso (promedio de todas sus materias). El ranking de un curso se obtiene ordenando los promedios generales de mayor a menor.

7.4 Gestión de Evidencias PDF

Los estudiantes pueden subir un archivo PDF (máximo 10 MB) como evidencia de cada actividad, siempre que la fecha actual esté dentro del rango fechaInicioEntrega - fechaFinEntrega. El archivo se recibe como multipart/form-data mediante Multer y se almacena en el campo archivoBytes (BYTEA) de la tabla evidencias en PostgreSQL. Adicionalmente se persisten metadatos como nombreArchivo, tamaño, tipoContenido y fechaSubida. La descarga se realiza vía GET /api/evidencias/:id/descargar. El borrado es un soft-delete: cambia el estado a ELIMINADO sin borrar el registro físico.

7.5 Notificaciones

El sistema genera notificaciones automáticamente al crear una actividad nueva (NUEVA_ACTIVIDAD) o al registrar una calificación (CALIFICACION). Las notificaciones se almacenan en la tabla notificaciones y son consultadas por el frontend mediante polling (TanStack Query con refetchInterval). El usuario puede marcarlas como leídas individualmente o en bloque.

8 Recomendaciones de Mejores Prácticas

Las siguientes recomendaciones aplican para el mantenimiento y extensión del sistema notas_v2:

- Nunca almacenar contraseñas, tokens ni cadenas de conexión en el código fuente. Usar variables de entorno y GitHub Secrets para secretos en CI/CD.
- Ejecutar `npx prisma migrate dev --name <descripcion>` ante cualquier cambio en el esquema. Nunca editar la base de datos manualmente en producción.
- Mantener los DTOs como fuente de verdad de validación. Cualquier nuevo campo debe declararse con sus decoradores `class-validator` antes de usarse en el servicio.
- Al agregar un nuevo endpoint protegido, aplicar siempre `@UseGuards(JwtAuthGuard, RolesGuard)` y `@Roles()` para evitar exposición accidental.
- Todos los formularios del frontend deben tener su esquema Zod correspondiente. No usar el estado del formulario sin validación.
- Al añadir un nuevo servicio de datos en el frontend, crear el `queryKey` correspondiente y usar `queryClient.invalidateQueries()` en las mutaciones para mantener el caché consistente.
- Documentar con JSDoc los métodos de servicio más complejos, especialmente los de cálculo de promedios y el validador de cédula.
- Antes de desplegar a producción, revisar que CORS esté restringido a los orígenes permitidos (actualmente `app.enableCors()` sin restricciones).
- Configurar protección de rama (branch protection) en la rama main del repositorio para exigir Pull Request y al menos una aprobación antes de hacer merge.
- Para escalar el sistema, considerar la extracción del almacenamiento de PDFs a un servicio de objetos (S3, MinIO) reemplazando el campo BYTEA, lo que reduciría el tamaño de la base de datos y facilitaría la purga de archivos.

/